ĐẠI HỌC ĐÀ NẴNG
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG VIỆT - HÀN
Vietnam - Korea University of Information and Communication Technology

Software Testing

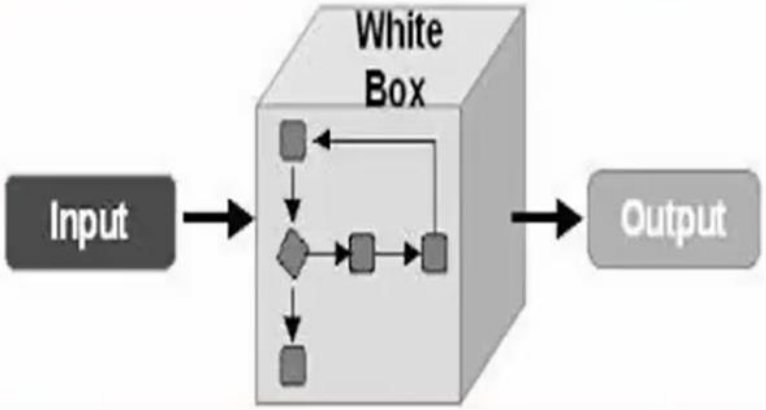
CHAPTER 3. SOFTWARE TESTING APPROACHES AND TECHNIQUES

Session 3

WHITE BOX TESTING

Contents

1.White box testing



- As known as:
  - Structural testing
  - Structure-based testing
  - Glass Box testing
- It is a testing in which testing based on the internal paths, structure and implementation of the software under test(SUT)
- The testing based on knowledge of the internal logic of an application's code, how it works
- White box testing is usually done by developers.

1.White box testing

- White-box testing techniques can be applied at all levels of system development but apply *primarily* to unit testing
- The major testing focuses on:
  - Program structures
    - Program statements and branches
    - Various kinds of program paths
  - Program internal logic and data structures
  - Program internal behaviors and states

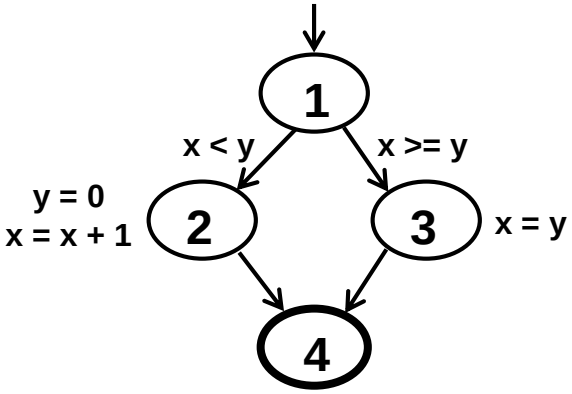
"A procedure to select test cases based on an analysis of the internal structure of a component or system"

2. Control Flow Graph

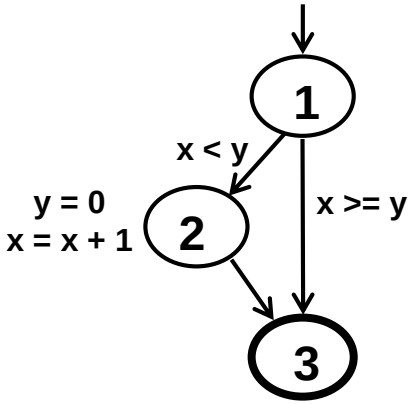
- A Control Flow Graph (CFG) is the graphical representation of control flow during the execution of programs or applications.
- Control Flow Graph components:
  - **Nodes** : Statements or sequences of statements (basic blocks). A predicate node has two edges leading out from it (True and False)
  - **Edges** : flow of control
  - **Basic Block** : A sequence of statements such that if the first statement is executed, all statements will be (no branches)
  - Regions: Areas bounded by a set of edges and nodes

CFG : The if Statement

```
if (x < y)
{
   y = 0;
   x = x + 1;
}
else
{
   x = y;
}
```

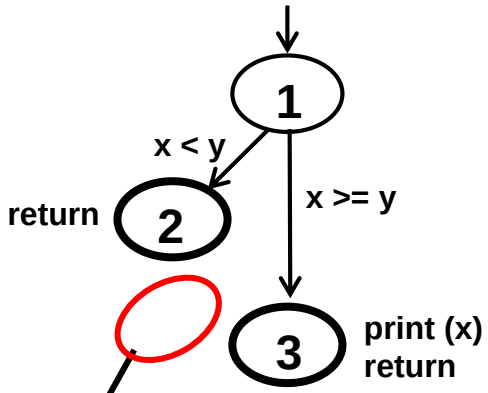



```
if (x < y)
{
   y = 0;
   x = x + 1;
}
```



CFG : The if-Return Statement

```
if (x < y)
{
   return;
}
print (x);
return;
```



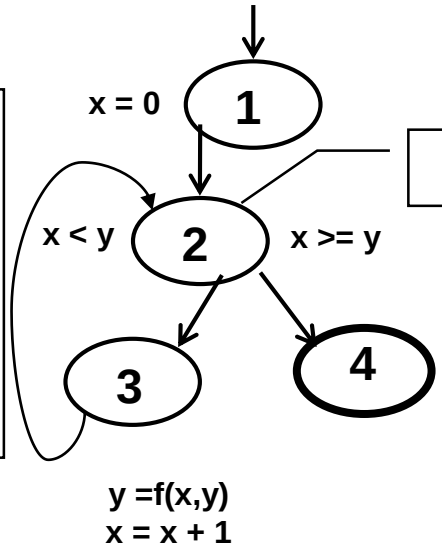
No edge from node 2 to 3.
The return nodes must be distinct.

CFG : Loop Statement

- Loops require "*extra*" nodes to be added
- Nodes that do not represent statements or basic blocks

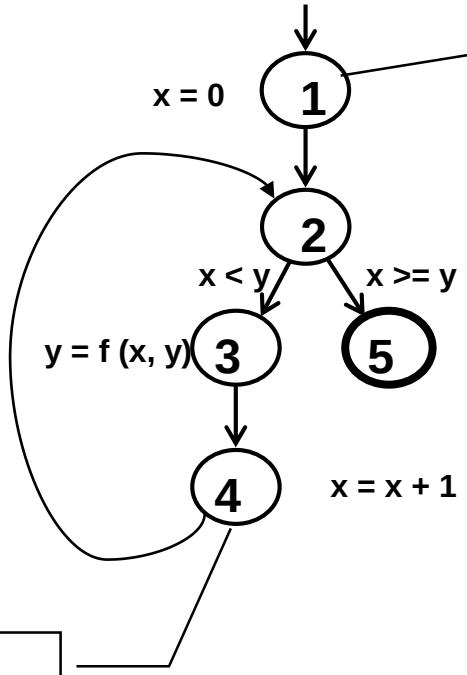
CFG : while and for Loops

```
x = 0;
while (x < y)
{
   y = f (x, y);
   x = x + 1;
}
```



dummy node

```
for (x = 0; x < y;
x++)
{
   y = f (x, y);
}
```

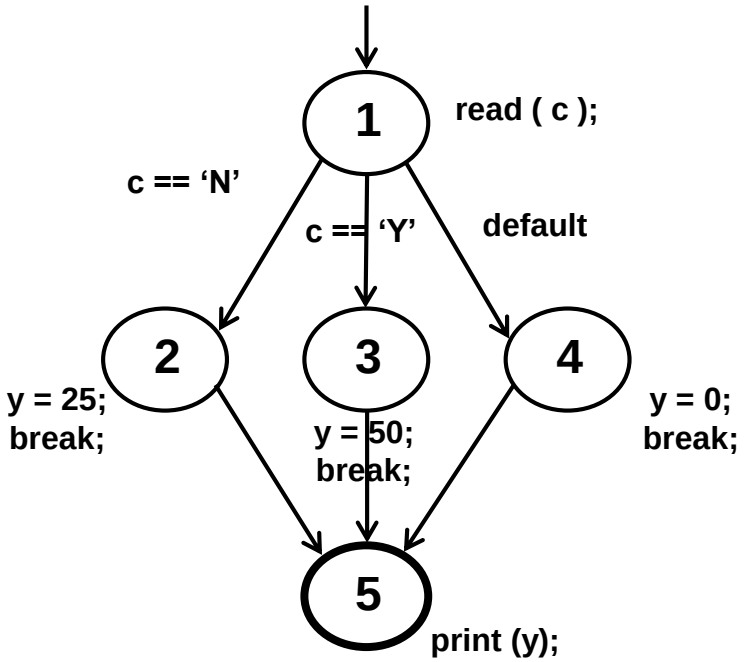


implicitly initializes loop

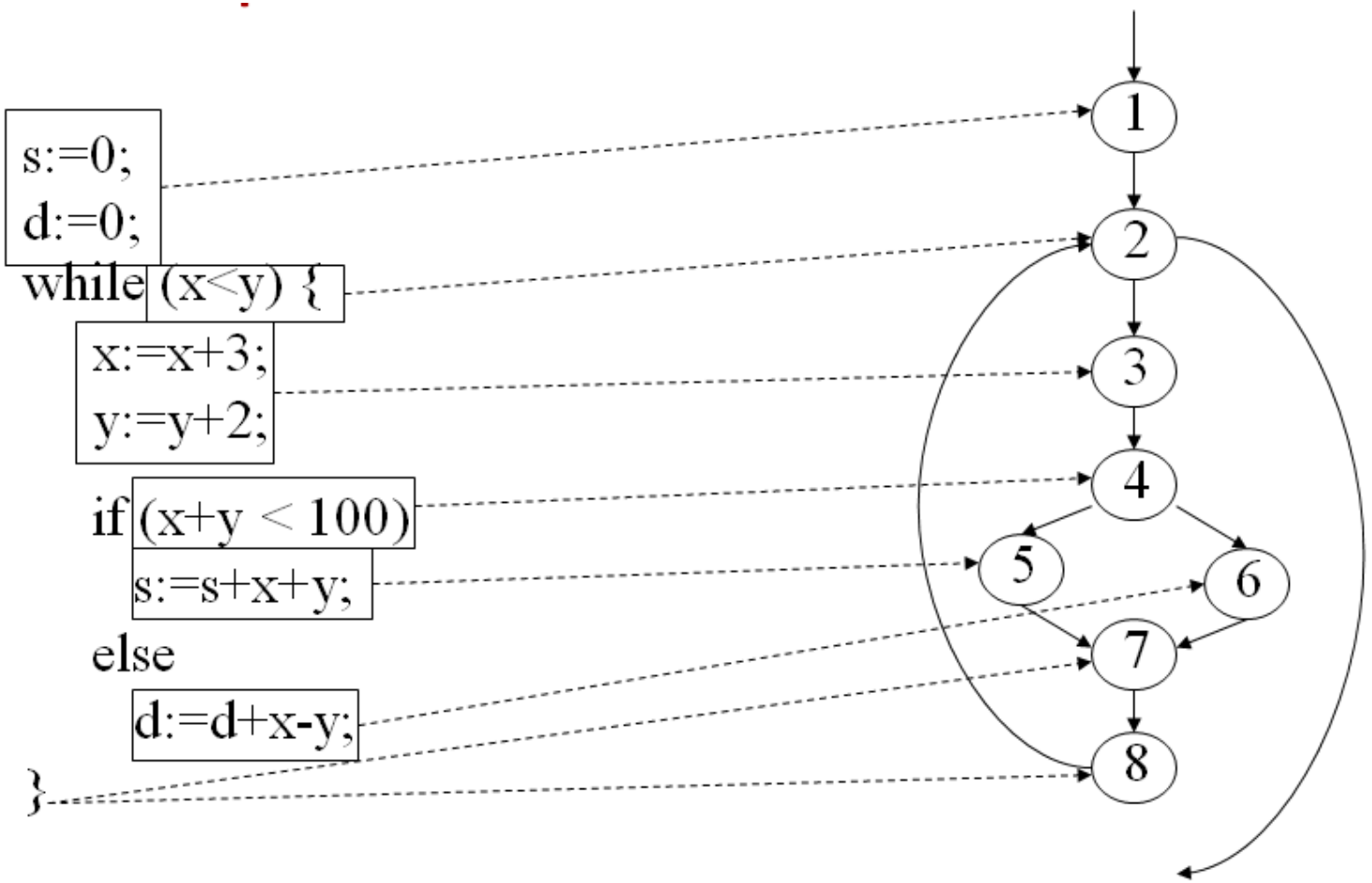
implicitly increments loop

CFG : The case (switch) Structure

```
read ( c) ;
switch ( c )
{
   case ‘N’:
      y = 25;
      break;
   case ‘Y’:
      y = 50;
      break;
   default:
      y = 0;
      break;
}
print (y);
```



CFG : Example



CFG : Example

❖Draw CFG

```
begin
   int x, y, power;
   float z;
   input (x, y);
   if (y<0)
     power=-y;
   else
     power=y;
   z=1;
   while (power!=0){
     z=z*x;
   power=power-1;
   }
   if (y<0)
     z=1/z;
   output(z);
end
```

CFG : Example

- Draw CFG

```
1. INPUT A
   INPUT B
   INPUT C
2. WHILE ( (A<20) )
3. PRINT A+B
4. IF (A==C)
5. PRINT A
6. ELSE
   PRINT C
7. END OF DO
8. IF(C<=100)
9 . PRINT A+B
10 . ELSE
     PRINT A-B
11 . END OF PROGRAM
```

CFG : Example

- Draw CFG

```
read A, B, C;
while (A > B) do
{
  if(A - B) > 100 then
      A = A - 10;
      B = B + 10
   else
      A--;
      B++;
  switch (C):
    case “Orange”: D = “Green”;
    case “Yellow”: D = “Red”;
    default: D = “Magenta”
}
write A, B, C, D;
```

CFG : Example

❖Draw CFG

```
1.  scanf("%d %d",&x, &y);
2.  If (y<0)
3.       pow = -y;
    else
4.  pow = y;
5.  Z = 1.0
6.  While (pow != 0) {
7.       z = z*x;
8.       pow = pow -1; }
9.  If (y <0)
10. z = 1.0 /z;
11. printf("%f",z);
```

3. White box testing techniques

- White box testing techniques consist of:
  - Statement coverage
  - Decision / Branch coverage
  - Condition coverage
  - Path coverage

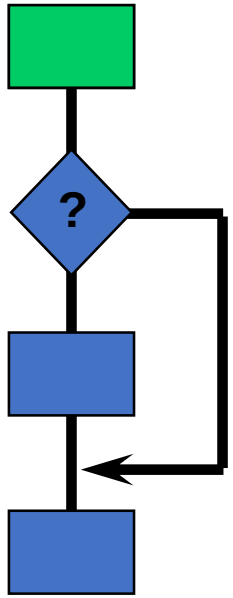
3.1 Statement coverage

- The statement coverage is also known as node coverage, line coverage or segment coverage, basic block coverage, C1 coverage.
- To achieve 100% statement coverage we have to make sure that **every statement in the code is executed at least once.**
- Black-box testing may actually achieve only 60% to 75% statement coverage. Typical ad hoc testing is likely to be around 30%.

3.1 Statement Coverage

- The statement coverage can be calculated as shown below:

$$\text{Statement coverage} = \frac{\text{Number of statements exercised}}{\text{Total number of statements}} \times 100\%$$



- Example
  - program has 100 statements
  - tests exercise 87 statements
  - statement coverage = 87%

Example 1

```
1  READ a
2  IF a > 6 THEN
3     b = a
4  ENDIF
5  PRINT b
```

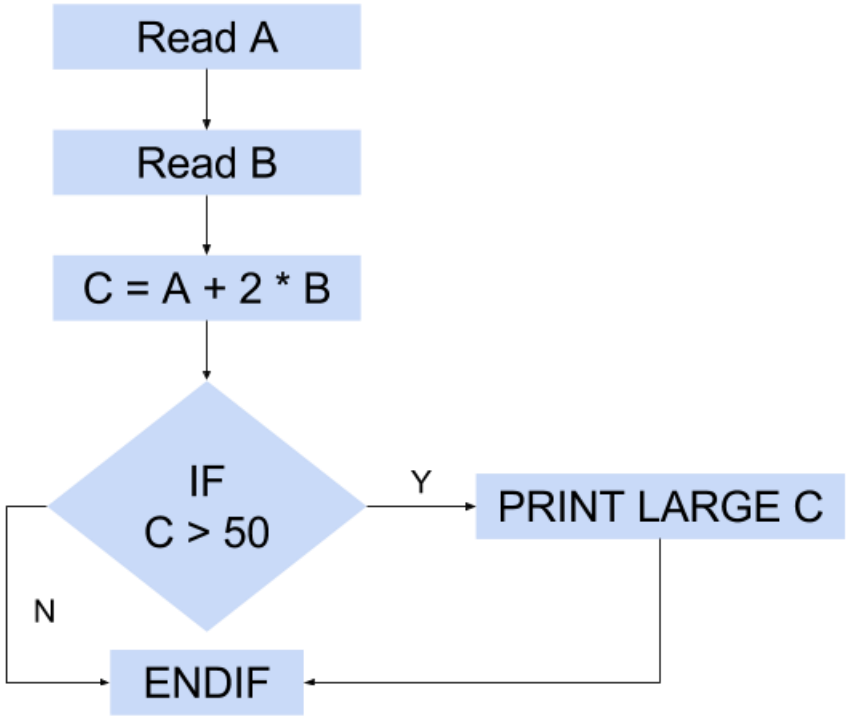
Test case	Input	Expected output
1	7	7

Statement numbers

As all 5 statements are ‘covered’ by this test case, we have achieved **100% statement coverage**

Example 2

```
1  READ A
2  READ B
3  C = A + 2 * B
4  IF C>50 THEN
5  PRINT  'LARGE C'
6  ENDIF
```



Test cases:

Test 1_1: A= 2, B = 3
Test 1_2: A= 0, B = 25
Test 1_3: A= 47, B = 1

83% statement coverage

Test 1_4: A = 20, B = 25 ***(C=70, 100% statement coverage)***

=> only one test case needed to achieve 100% statement coverage

Example 3

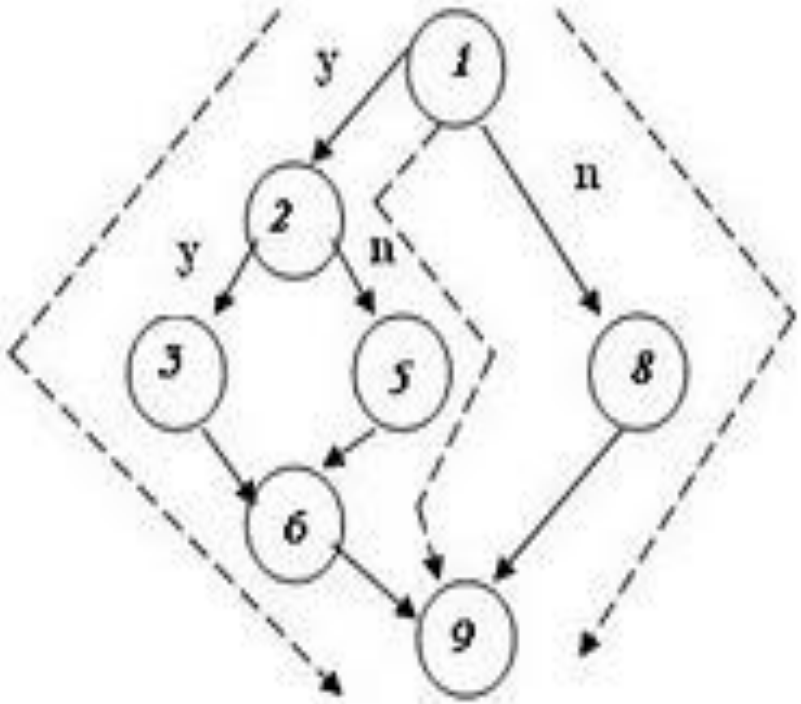
- Compute statement coverage?

```
1  READ A
2  READ B
3  IF A>B THEN
4       PRINT  'A is greater than B'
5  ELSE
6       PRINT 'B is greater than A'
7  ENDIF
```

Example 4

❖How many tests are required to achieve 100% statement coverage?

```
1. If fuel tank empty
2.       If petrol engine
3.                Fill with petrol
4.       Else
5.                Fill with diesel
6.       Elseif
7. Else
8.        Endif
```



3.2 Decision coverage

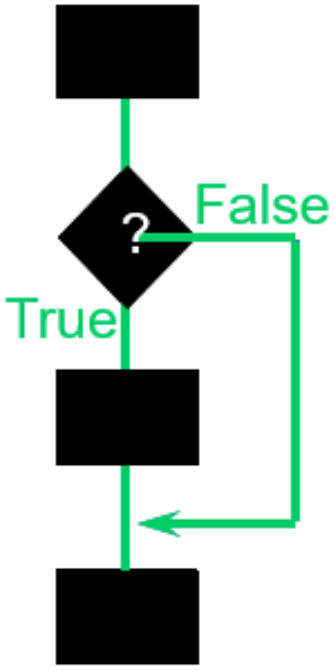
- Also known and Branch Coverage and C2 Coverage.
- This technique is used to make sure that **all branch** of every control structure (if, while, etc.) **is executed at least once.**
- It **covers both the true and false conditions** unlikely the statement coverage.
- Disadvantage: Does not cover all conditions in logical operators.

100% decision coverage guarantees 100% statement coverage, but 100% statement coverage does not guarantees 100% decision coverage

3.2 Decision coverage

- The decision coverage can be calculated as given below:

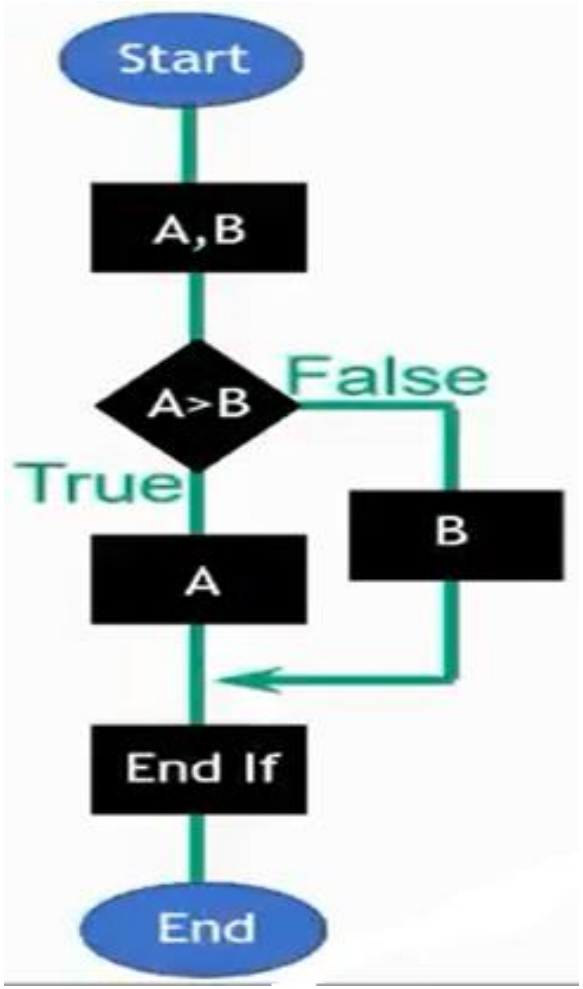
$$\text{Decision coverage} = \frac{\text{Number of decision outcomes exercised}}{\text{Total number of decision outcomes}} \times 100\%$$



- Example:
  - program has 120 decision outcomes
  - tests exercise 60 decision outcomes
  - decision coverage = 50%

Example 1

```
1 READ A
2 READ B
3 IF A>B THEN
4    PRINT 'A is bigger'
5 ELSE
6    PRINT 'B is bigger'
7 ENDIF
```



TEST CASES

Test 1_1: **A = 20, B = 15**

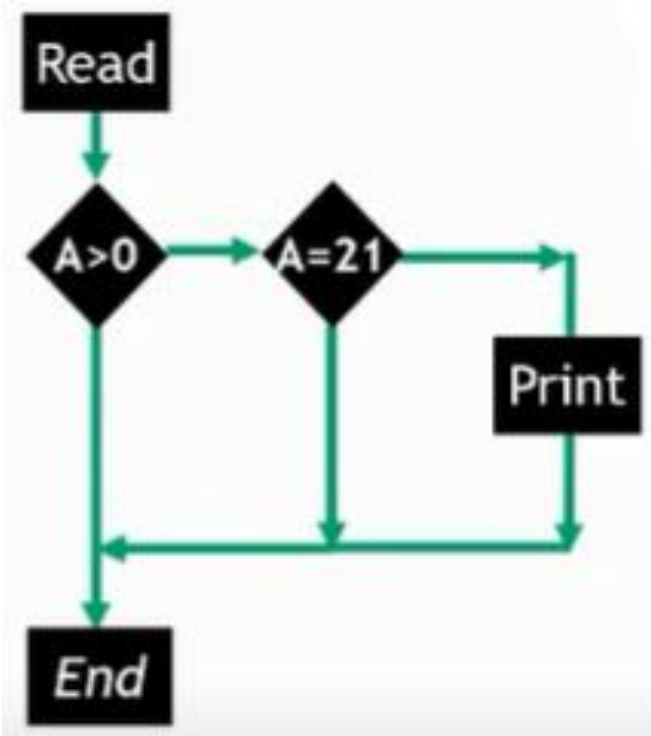
Test 1_2: **A = 10, B = 15**

This now covers both of the decision outcomes: True (with Test 1_1) and False (with Test 1_2).

Decision coverage = 2

Example 2

```
1 READ A
2 IF A>0 THEN
3    IF A = 21 THEN
4        PRINT 'A = 21'
5    ENDIF
6 ENDIF
```



What is the minimum number test cases required for 100% decision coverage?

Decision coverage= ?

Example 3

❖Consider the following Pseudo code:

```
IF (A>B)
    THEN IF (A>C)
        THEN IF (A>D)
            THEN IF (A>E)
                THEN Print "A is large."
            End IF
        End IF
    End IF
End IF
```

❖How many minimum test cases are required to cover 100% statement coverage and decision coverage?

A. 2 for Statement, 5 For Decision

B. 1 for Statement, 2 for Decision

C. 1 for Statement, 5 for Decision

D. 5 for Statement, 1 for Decision

3.3 Condition coverage

- Is known as Predicate Coverage
- This technique is used to make sure that each one of the Boolean expression have been evaluated to both TRUE and FALSE.
  - True at least once and
  - False at least once.

$$Condition\ Coverage = \frac{Number\ of\ Executed\ Operands}{Total\ Number\ of\ Operands}$$

Note: 100% condition coverage does not guarantee 100% decision coverage.

Example

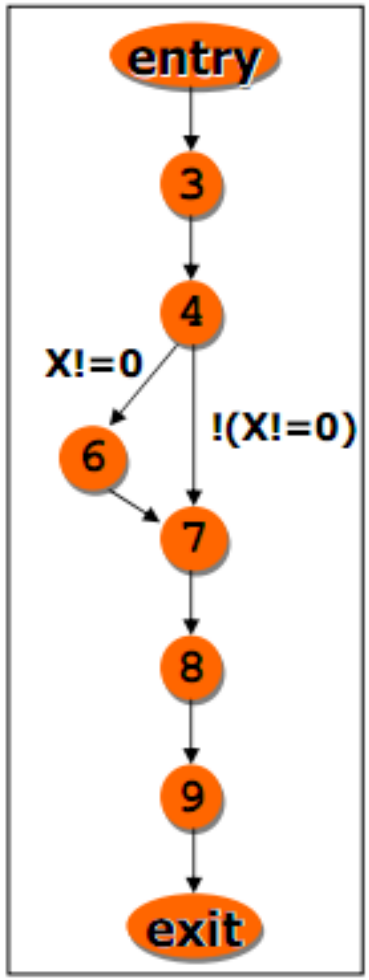
```
1 IF (A && B) THEN
2    PRINT C
3  ENDIF
```

There are 2 test cases for condition coverage
TEST 1: A=TRUE, B=FALSE
TEST 2: A=FALSE, B=TRUE

Example 2

❖Identify test cases for statement coverage?

```
1. void main() {
2.     float x, y;
3.     read(x);
4.     read(y);
5.     if (x!=0)
6.         x = x+10;
7.     y = y/x;
8.     write(x);
9.     write(y);
10. }
```



❖Test requirements

3,4,6,7,8,9

❖Test specification

(x!=0, any y)

❖Test cases

(x=20, y=30)

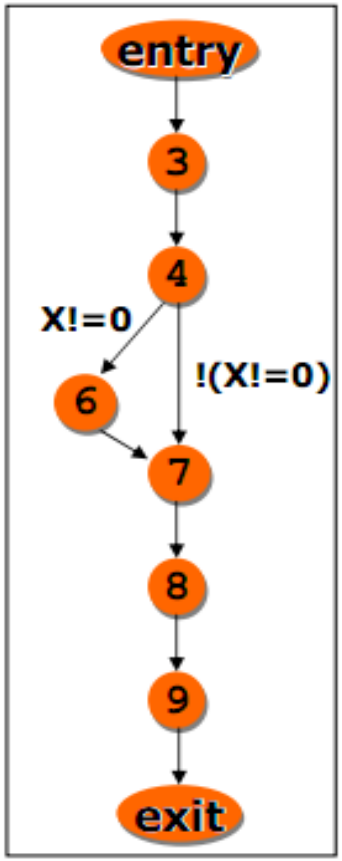
Such test does not reveal the fault at statement 7

=>To reveal it, we need to traverse edge 4-7

⇒ branch coverage

Example 2 (tt)

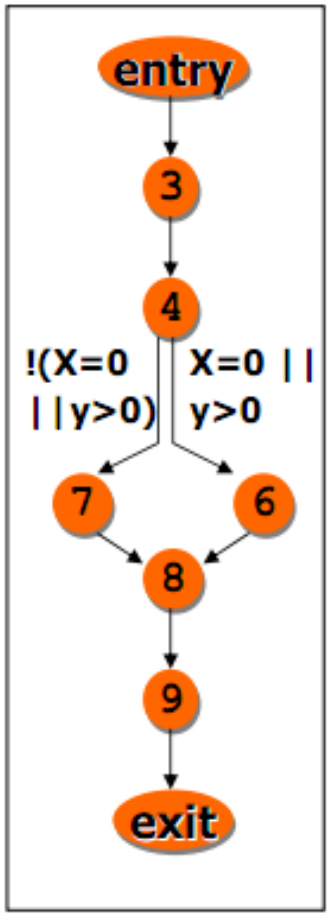
```
1. void main() {
2.     float x, y;
3.     read(x);
4.     read(y);
5.     if (x!=0)
6.         x = x+10;
7.     y = y/x;
8.     write(x);
9.     write(y);
10. }
```



- Test requirements
  - Edges 4-6 and 4-7
- Test specification
  - (x!=0, any y)
  - (x=0, any y)
- Test cases
  - (x=20, y=30)
  - (x=0, y=30)

Example 3

```
1. void main() {
2.   float x, y;
3.   read(x);
4.   read(y);
5.   if(x==0)||(y>0)
6.     y = y/x;
7.   else x = y+2;
8.   write(x);
9.   write(y);
10.}
```



- Consider test cases

  (x=5,y=5),

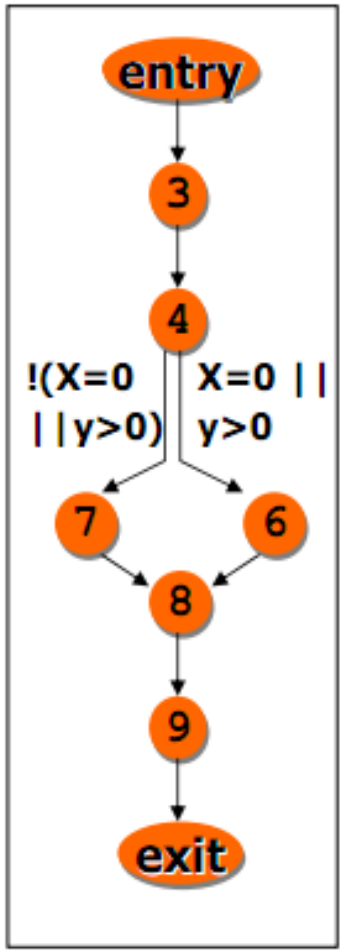
  (x=5, y=-5)

- The test suite is adequate for branch coverage, but does not reveal the fault at statement 6
- Predicate 4 can be true or false operating on only one condition

⇒ Basic condition coverage

Example 3 (tt)

```
1. void main() {
2.   float x, y;
3.   read(x);
4.   read(y);
5.   if(x==0)||(y>0)
6.     y = y/x;
7.   else x = y+2;
8.   write(x);
9.   write(y);
10.}
```



- Consider test cases
  (x=0,y=-5),
  (x=5, y=5)
- The test suite is adequate for basic condition coverage, but it does not reveal the fault at statement 6
- The test suite is not adequate for branch coverage.

⇒Branch and condition coverage

Branch and Condition Coverage

- Branch + Condition Coverage requires that both Branch AND Condition Coverage will have been achieved.
- Branch + Condition Coverage subsumes both Branch Coverage and Condition Coverage.
- Example:

```
if ( ( a || b ) && c ) { … }
```

a	b	c	Outcome
T	T	T	T
F	F	F	F

Compound Condition Coverage

- Is known as Multiple condition coverage
- Design test cases for each combination of conditions
- All combinations of condition values at every branch statement covered (and every entry point taken) This would result in 2^n tests, given n conditions.

Compound Condition Coverage

- Example:

```
1 IF (A || B) THEN
2    PRINT C
3  ENDIF
```

There are two Boolean Expressions A and B, so the Compound Condition Coverage is defined as:
Test Case1: A=TRUE, B=TRUE
Test Case2: A=TRUE, B=FALSE
Test Case3: A=FALSE, B=TRUE
Test Case4: A=FALSE, B=FALSE

Exercise

```
private void GetTriangle (int a, int b, int c)
 {
   if ((a + b <= c) || (b + c <= a) || (a + c <= b))
                {
                    // Error
                }
        else if ((a == b) && (b == c))
                {
                    // Equilateral;
                }
 else if ((a == b) || (b == c) || (a == c))
                {
                    // Isoceles;
                }
                else
                {
                    // Scalene;
                }
 }
```

Let's

- Graph the control flow!
- Compute coverage and design test cases

3.4 Path Coverage

- This technique is used to **testing all possible paths** which means that each statement and branch is covered at least once.
- Strongest white-box criterion (based on control flow analysis)
- Enables the test case designer to derive a logical complexity measure of a procedural design
- Uses this measure as a guide for defining a basis set of execution paths

3.4 Path Coverage

- In path testing method, the control flow graph (CFG) of a program is designed to find a set of linearly independent paths of execution.
- **McCabe's Cyclomatic Complexity** is used to determine the number of linearly independent paths and then test cases are generated for each path.

3.4 Path Coverage

- In order to reduce the redundant tests and to achieve maximum test coverage, basis path testing is used.
- Advantages of Basic Path Testing
  - It helps to reduce the redundant tests
  - It focus attention on program logic
  - It defines the number of independent paths to write test cases needed to ensure that every statement and condition will be executed at least one time.

4.Cyclomatic Complexity

- Cyclomatic Complexity (CC) of the flow chart indicates the number of independent paths of the program.
- CC gives an indication of the minimum number of test cases required
- It is said to be a measure of the logical complexity of a program

4.Cyclomatic Complexity

❖Cyclomatic complexity (CC) can be calculated using 3 different formulae:

1. Based on edges and nodes:

$$CC = E - N + 2P,$$

where:

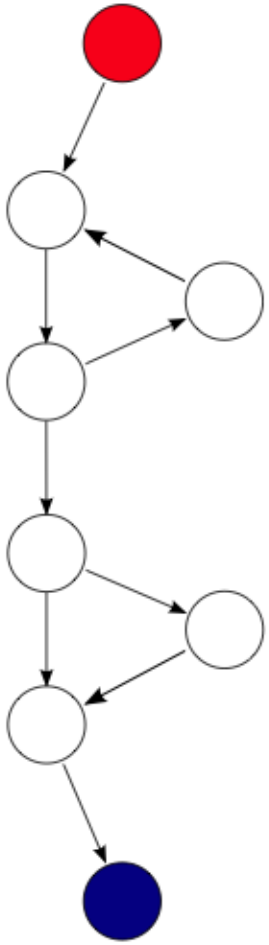
E = the number of edges of the graph

N = the number of nodes of the graph

P = the number of connected components

2. Number of closed regions (including the surrounding rectangle) +1
3. Number of binary predicate + 1.

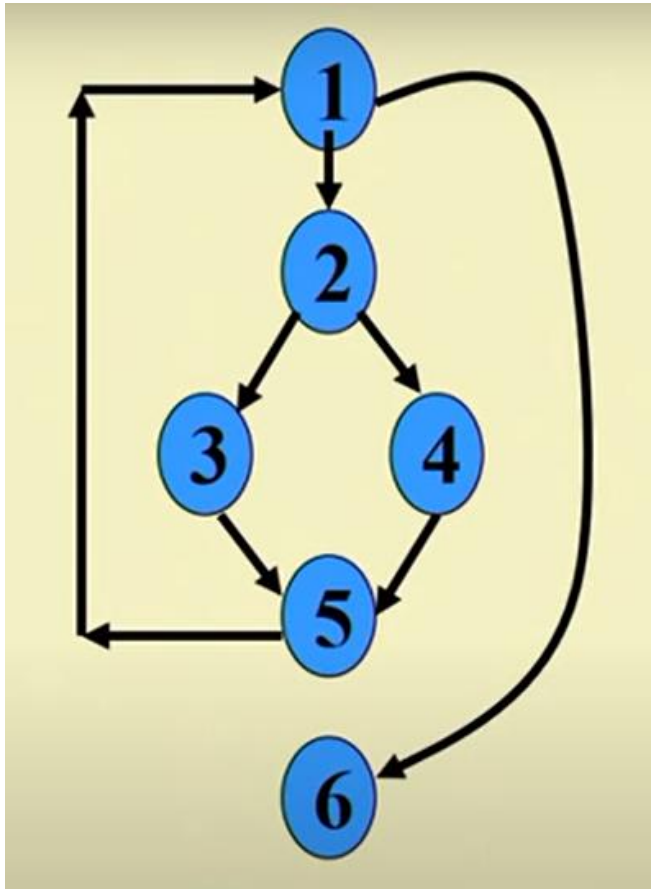
Example 1



CC = 3:

- ✓ 9 (edges) – 8 (nodes) + 2*1 (connected component) = 3
- ✓ 2 (closed regions) +1 = 3
- ✓ 2 (Conditional Nodes) + 1 = 3

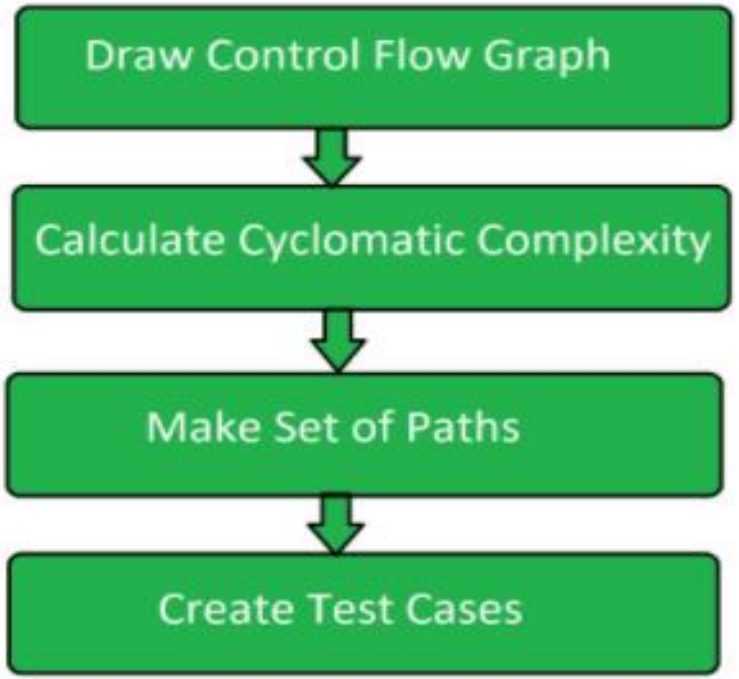
Example 2



Compute CC?

5. Basic Path Testing

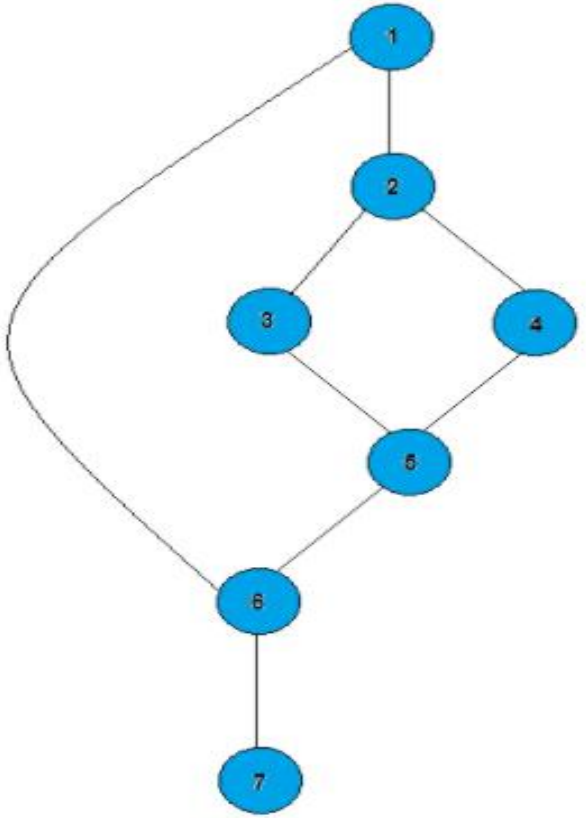
- The basic steps involved in basis path testing include:



Basis Path Testing: Example

❖**Step 1: Draw a control flow graph**

```
1: IF A = 100

2: THEN IF B > C

3: THEN A = B

4: ELSE A= C

5: ENDIF

6: ENDIF

7: Print A
```



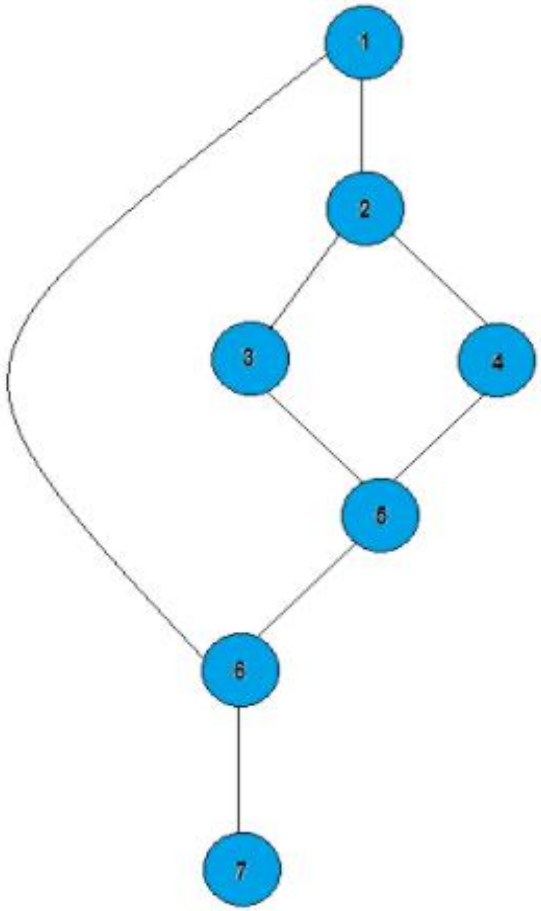
Basis Path Testing: Example

❖Step 2: Calculate cyclomatic complexity (CC)

CC = edges - nodes + 2p = 8-7+2*1=3

CC = number of closed regions+1 = 2+1 =3

CC = number of binary predicate + 1 = 2+1 =3



Basis Path Testing: Example

- Step 3: Make set of basis paths

The cyclomatic complexity tells us the number of paths to evaluate for basis path testing.

We have 3 paths and our basis set of paths is:

- **Path 1**: 1, 2, 3, 5, 6, 7.
- **Path 2**: 1, 2, 4, 5, 6, 7.
- **Path 3**: 1, 6, 7.

Basis Path Testing: Example

❖Step 4: Create test cases

After determining the basis of path, we will generate the test case for each path.

Usually we need at least 3 test cases to cover 3 paths, but in this example Path 3 is already covered by Path 1 and 2 so we just write 2 test cases only.

Test case	From Path	A	B	C	Expected result
1	P1	100	120	50	A=120
2	P2	100	20	50	A=20
3	P3	90	-	-	A=90

Exercise 1

❖Basis path testing for the segment code:

```
   public double calculate(int amount)
    {
1. double rushCharge = 0;
    if (nextday.equals("yes") )
     {
2.       rushCharge = 14.50; }
3  double tax = amount * .0725;
3  if (amount >= 1000) {
4.       shipcharge = amount * .06 + rushCharge; }
5. else if (amount >= 200) {
6.       shipcharge = amount * .08 + rushCharge; }
7.  else if (amount >= 100) {
8.       shipcharge = 13.25 + rushCharge; }
9.  else if (amount >= 50)  {
10.       shipcharge = 9.95 + rushCharge; }
11. else if (amount >= 25) {
12.       shipcharge = 7.25 + rushCharge; }
     else {
13.      shipcharge = 5.25 + rushCharge; }
14. total = amount + tax + shipcharge;
14. return total;
} //end calculate
```

Exercise 2

- Basis path testing for the segment code:

```
internal static int GetMaxDay(int month, int day, int year)
    {
        int maxDay = 0;
      if (!IsInvalidDate(month, day, year)) {
        if (IsThirtyOneDayMonth(month))  {
            maxDay = 31;
          }
        else if (IsThirtyDayMonth(month))    {
            maxDay = 30;
          }
          else    {
            maxDay = 28;
         if (IsLeapYear(year))    {
              maxDay = 29;
            }
          }

        return maxDay;
  }
```

ĐẠI HỌC ĐÀ NẴNG
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG VIỆT - HÀN
Vietnam - Korea University of Information and Communication Technology

Thank You !